GSwifT: Delay-Based Congestion Control on Linux and Programmable Hardware
Members: Ethan Kim, Gregory Park, Niki Hu, Xiaomeng Ye, Raymond Zhao

Swift is Google's delay-based congestion control algorithm. It achieves low latency and high throughput by precisely measuring round-trip delay and separating network delay from endpoint delay. However, it relies on Google's specialized infrastructure, including NIC hardware timestamps, an OS-bypass transport stack, and fine-grained pacing. We ask how much of Swift's benefit can be reproduced on commodity Linux. We will implement Swift as an eBPF congestion control module and test whether three key capabilities carry over: (1) accurate delay measurement using software timestamps, (2) distinguishing network congestion from receiver congestion, and (3) handling large incasts by sending less than one packet per round trip. We will evaluate latency, throughput, fairness, and incast behavior on an emulated datacenter network built with Linux network namespaces, comparing against Cubic and DCTCP and checking whether the Swift paper's trends still hold. As a stretch goal, we will run Swift between two machines with hardware-timestamping NICs to measure how much performance software timing costs.

Our work builds on three papers. *Swift: Delay is Simple and Effective for Congestion Control in the Datacenter* introduces the core design and shows that splitting delay into network and endpoint components lets each source of congestion be handled separately. *Congestion Control for Spraying with Congested Paths* introduces MSwift, which makes Swift more robust when packets take paths with different delays, by tolerating reordering and reacting to the median of recent delay measurements. *Falcon: A Reliable, Low Latency Hardware Transport* builds Swift-style congestion control into a hardware transport with precise timing and pacing. Together, these show Swift works well with sophisticated infrastructure; we ask whether similar benefits are achievable on widely available platforms.